

Generative AI

Lab 5

Can a Transformer Generate Good Quality Code?

Student: Meryem ElBiach

Programming Language: Java

Environment: Google Colab

1. Research Question and Hypothesis

1.1. Research Question

The objective of this experiment is to investigate how the selected Transformer model affects the quality of generated Java source code across programming tasks with different levels of difficulty.

The research question is:

How does the choice of Transformer model affect the quality of Java code generated for programming tasks of different difficulty levels?

1.2. Hypothesis

Before running the experiment, the following hypothesis was defined:

Code-specialized and instruction-tuned Transformer models are expected to generate more correct and robust Java code than a general-purpose Transformer model, particularly as task difficulty increases.

2. Experimental Design

2.1. Selected Programming Language

Java was selected as the programming language for all tasks, prompts, generated programs, and test cases.

Using the same language throughout the experiment ensures that the programming language remains a controlled variable.

2.2. Experimental Variables

Variable Type	Definition in this Experiment
Independent variable	The Transformer model used for code generation.
Dependent variables	Java validity, compilation success, functional correctness, robustness, and instruction following.
Controlled variables	Programming language, task definition, prompt meaning, test cases, generation settings, and evaluation procedure.

Table 1: Experimental variables.

3. Selected Transformer Models

Three models were selected to compare different levels of specialization: a general-purpose language model, a code-specialized model, and a code-oriented instruction-tuned model.

Model	Model ID	Role in Experiment
DistilGPT2	distilbert/distilgpt2	General-purpose baseline.
CodeGen-Multi 350M	Salesforce/codegen-350M-multi	Code-specialized model used to evaluate source-code generation performance.
Qwen2.5-Coder 0.5B Instruct	Qwen/Qwen2.5-Coder-0.5B-Instruct	Code-oriented instruction-tuned model used to evaluate both code generation and instruction following.

Table 2: Transformer models evaluated in the experiment.

4. Programming Tasks

Three original programming tasks were designed with increasing difficulty, from basic conditional logic to array processing and edge-case handling.

Task	Name	Difficulty	Main Skills Tested
1	Even or Odd Number	Easy	Basic Java syntax, input handling, conditional statements, and the modulo operator.
2	Maximum Value in an Array	Medium	Array manipulation, loops, input processing, and comparisons.
3	Second Largest Distinct Number	Challenging	Arrays, duplicate handling, negative values, edge cases, and more complex reasoning.

Table 3: Programming tasks used in the experiment.

5. Prompts

The same prompt was used for all three Transformer models for each task. No model-specific improvement was introduced after observing the initial results.

5.1. Task 1 Prompt

Generate valid Java source code.

Task:

Create a program that determines whether an integer is even or odd.

Requirements:

- Use a class named Main.
- Read one integer from standard input.
- Print exactly "Even" if the number is even.
- Print exactly "Odd" if the number is odd.
- Do not print explanations or additional text.
- Return source code only.

5.2. Task 2 Prompt

Generate valid Java source code.

Task:

Create a program that finds the maximum value in an integer array.

Requirements:

- Use a class named Main.
- Read an integer n representing the array size.
- Read n integers.
- Print only the maximum value.
- Do not print explanations or additional text.
- Return source code only.

5.3. Task 3 Prompt

Generate valid Java source code.

Task:

Create a program that finds the second largest distinct value in an integer array.

Requirements:

- Use a class named Main.
- Read an integer n representing the array size.
- Read n integers from standard input.
- Print only the second largest distinct value.
- If there is no second distinct value, print "No second value".
- Do not print explanations or additional text.
- Return source code only.

6. Predefined Test Cases

The test cases were defined before code generation.

6.1. Task 1 – Even or Odd

Test	Input	Expected Output
1	4	Even
2	7	Odd
3	0	Even
4	-3	Odd

Table 4: Predefined test cases for Task 1.

6.2. Task 2 – Maximum Value

Test	Input	Expected Output
1	5 / 3 8 2 10 4	10
2	4 / -5 -2 -9 -1	-1
3	1 / 7	7
4	5 / 4 4 4 4 4	4

Table 5: Predefined test cases for Task 2.

6.3. Task 3 – Second Largest Distinct Value

Test	Input	Expected Output
1	5 / 3 8 2 10 4	8
2	6 / 5 5 4 4 3 3	4
3	4 / 7 7 7 7	No second value
4	5 / -10 -3 -7 -3 -5	-5

Table 6: Predefined test cases for Task 3.

7. Experimental Results

All generated outputs were preserved, including incomplete and incorrect responses. Programs containing recognizable Java source code were compiled and executed against the predefined test cases.

Task	Model	Java	Compile	Correct	Robust	Main Observation
1	DistilGPT2	Fail	Fail	Fail	Fail	Repeated prompt text instead of generating Java code.
1	CodeGen 350M	Fail	Fail	Fail	Fail	Generated descriptive text rather than a complete Java program.
1	Qwen2.5-Coder	Pass	Pass	Fail	Fail	Correct even/odd logic, but additional input text caused all exact-output tests to fail.
2	DistilGPT2	Fail	Fail	Fail	Fail	No valid Java class was generated.
2	CodeGen 350M	Fail	Fail	Fail	Fail	No valid Java program was generated.
2	Qwen2.5-Coder	Pass	Pass	Fail	Fail	Used command-line arguments instead of standard input, causing runtime exceptions.
3	DistilGPT2	Fail	Fail	Fail	Fail	Repeated instructions rather than producing source code.
3	CodeGen 350M	Fail	Fail	Fail	Fail	Generated Python instead of Java and produced incomplete code.
3	Qwen2.5-Coder	Pass	Pass	Fail	Fail	The algorithm produced the expected values, but additional input and output messages violated the required format.

Table 7: Summary of experimental results.

8. Detailed Observations

8.1. DistilGPT2

DistilGPT2 did not generate executable Java code for any of the three tasks. Its outputs mainly repeated parts of the prompt such as **Return source code only**. Therefore, the outputs could not be compiled or functionally tested.

8.2. CodeGen-Multi 350M

CodeGen-Multi also failed to generate valid Java programs in the tested conditions. For Task 1 and Task 2, it mainly generated descriptions, input/output formats, or repeated text.

For Task 3, it generated Python code despite the explicit Java requirement. This was classified as a language and instruction-following failure.

8.3. Qwen2.5-Coder 0.5B Instruct

Qwen2.5-Coder generated compilable Java programs for all three tasks and usually captured the intended algorithm.

However, it repeatedly violated formatting and input/output constraints.

For Task 1, the algorithm correctly classified even and odd numbers, but the program printed an additional prompt:

```
Enter an integer: Even
```

instead of the required:

```
Even
```

For Task 2, the generated program used:

```
args[0]
```

instead of standard input. Since the predefined tests supplied data through standard input, the program crashed with an `ArrayIndexOutOfBoundsException`.

For Task 3, the algorithm correctly found the second largest distinct value, including duplicate and negative-value cases. However, the program printed additional messages such as:

```
The second largest distinct value is: 8
```

instead of printing only:

```
8
```

9. Failure Analysis

One particularly informative failure occurred with Qwen2.5-Coder on Task 2.

The model generated valid Java code and the logic for finding the maximum value was reasonable. However, it did not follow the requested input method. Instead of reading values from standard input, it tried to read them using `args[]`.

Because the predefined tests sent the values through standard input, the program received no command-line arguments and crashed with:

```
java.lang.ArrayIndexOutOfBoundsException
```

The main issue was therefore not the maximum-finding algorithm itself, but the fact that the model misunderstood an important instruction about how input should be provided.

This failure can primarily be classified as an **instruction-following failure**, which then caused a **runtime failure**.

10. Scientific Interpretation

The experiment showed a clear difference in behavior between the three tested models under the selected conditions.

DistilGPT2, used as the general-purpose baseline, did not produce usable Java source code for any of the three tasks.

CodeGen-Multi 350M, despite being specialized for code generation, also failed to produce valid Java programs under the instruction-style prompts used in this experiment. In Task 3, it generated Python rather than Java.

Qwen2.5-Coder 0.5B Instruct consistently generated compilable Java programs and generally captured the intended algorithm. However, none of its initial programs passed all exact-output tests because it did not consistently respect input/output and formatting constraints.

These observations suggest that, in this experiment, instruction following was as important as algorithmic correctness. A model may understand the requested computation but still fail an objective test if it ignores constraints such as the required input mechanism or exact output format.

11. Hypothesis Evaluation

The initial hypothesis was partially supported.

Qwen2.5-Coder, which combines code specialization with instruction tuning, generated more usable Java code than the general-purpose DistilGPT2 baseline.

However, the expectation that every code-specialized model would perform better was not supported. CodeGen-Multi 350M did not generate executable Java solutions under the tested prompting conditions.

The observations therefore indicate that code specialization alone was not sufficient in this experiment. The ability to interpret and follow natural-language instructions also appeared to play an important role.

12. Limitations

Several limitations prevent stronger general conclusions:

- Only three Transformer models were evaluated.
- Only three Java programming tasks were used.
- Each model-task combination was generated only once.
- Generation used deterministic settings (`do_sample=False`), so alternative decoding strategies were not evaluated.
- The selected models differ in size, training data, specialization, and instruction tuning. Therefore, the observed differences cannot be attributed to Transformer architecture alone.
- Exact-output evaluation is strict. A logically correct program may still fail if it prints additional text.
- The experiment was executed in Google Colab using CPU during the recorded runs.

13. Possible Future Experiment

A useful follow-up experiment would repeat every model-task combination several times and compare different prompting strategies while keeping the programming tasks fixed.

Another possible experiment would compare the current prompts with more explicit format-constrained prompts to determine whether instruction-following errors can be reduced.

14. Conclusion

This experiment evaluated three Transformer models for Java code generation across easy, medium, and more challenging programming tasks.

Under the tested conditions, DistilGPT2 and CodeGen-Multi 350M did not produce executable Java programs. Qwen2.5-Coder 0.5B Instruct generated compilable Java code and generally understood the intended algorithms, but repeatedly violated input/output and formatting constraints.

The results show that code-generation quality cannot be evaluated only by checking whether the underlying algorithm appears correct. Compilation, runtime behavior, robustness, and instruction following are also important evaluation dimensions.

Within this limited experiment, instruction tuning was associated with more usable code generation. However, because only three models, three tasks, and one generation per model-task combination were evaluated, this observation should not be generalized beyond the experimental conditions used here.